

Tempo

Réserver un bureau ou une salle

Titre professionnel **Concepteur développeur d'applications**
RNCP 37873

Valentin Musset · 3W Academy
Soutenance du 7 octobre 2026

Sommaire

1. Mon parcours et le projet
2. La conception
3. L'organisation du travail
4. L'architecture
5. Une réservation, du formulaire à la base
6. La sécurité
7. Les tests et la qualité
8. La CI et le déploiement
9. Le bilan et les suites
10. Conclusion

1. Introduction

Valentin Musset

- Diplôme **Développeur web full stack**, 3W Academy, obtenu en 2025
- Préparation du titre CDA en alternance chez **Collectif Energie**
- Tempo : mon projet personnel de certification, réalisé en dehors de l'entreprise

1.1. Contexte

Des espaces partagés, des réservations à coordonner

- Un bureau ou une salle peut recevoir plusieurs demandes pour le même créneau
- Un planning partagé gère mal les invitations et le suivi de présence
- Tempo s'adresse aux entreprises, écoles et centres de formation qui partagent leurs espaces

1.2. Objectif du projet

Collaborateur

- Réserver un espace
- Inviter ou rejoindre des participants
- Confirmer sa présence par QR code

Administrateur

- Gérer les espaces et les comptes
- Consulter toutes les réservations
- Suivre l'occupation et les suppressions

1.3. Contraintes

Métier	Technique et organisation
Aucun chevauchement pour un espace	Contrôles dans l'API et PostgreSQL
Une invitation en attente occupe une place	Gérer les demandes simultanées
Des droits différents selon le compte	Vérifier les droits côté serveur
Un projet personnel en parallèle de l'alternance	Avancer par fonctionnalités dans Trello

2. Conception

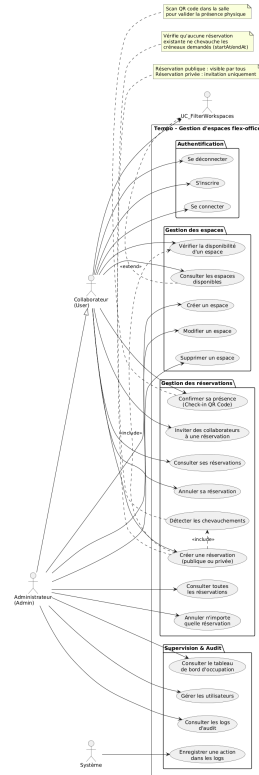
Du besoin aux modèles de données

2.1. Démarche de conception

1. Décrire les rôles et les règles dans `SPECS.md`
2. Préparer les écrans avec Figma
3. Modéliser les parcours en UML et les données avec MERISE
4. Traduire ces choix en routes, services et migrations SQL

Périmètre V1 : comptes, espaces, réservations, participants et check-in

2.2. Diagramme de cas d'utilisation



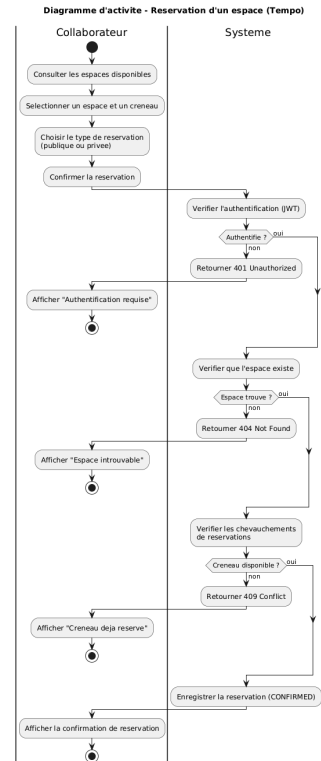
Deux rôles à distinguer

- Le collaborateur gère ses réservations
- L'administrateur dispose aussi des fonctions de gestion

Les droits sont contrôlés côté serveur. Le check-in confirme une action pendant le créneau, sans prouver la présence physique.

Ouvrir le diagramme complet dans un nouvel onglet

2.3. Diagramme d'activité



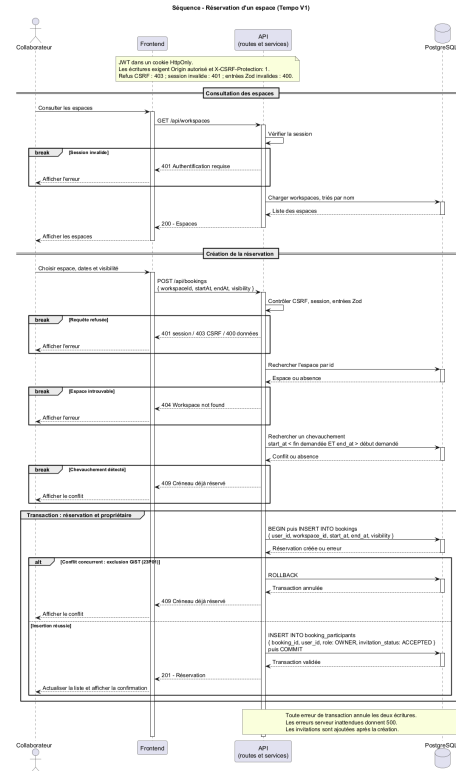
Parcours retenu : réserver un espace

Choisir un espace et un créneau,
vérifier la disponibilité, puis enregistrer
la réservation.

Dans la V1, les dates sont des instants complets
et la contrainte PostgreSQL protège aussi les
demandes simultanées.

Ouvrir le diagramme complet dans un nouvel onglet

2.4. Diagramme de séquence



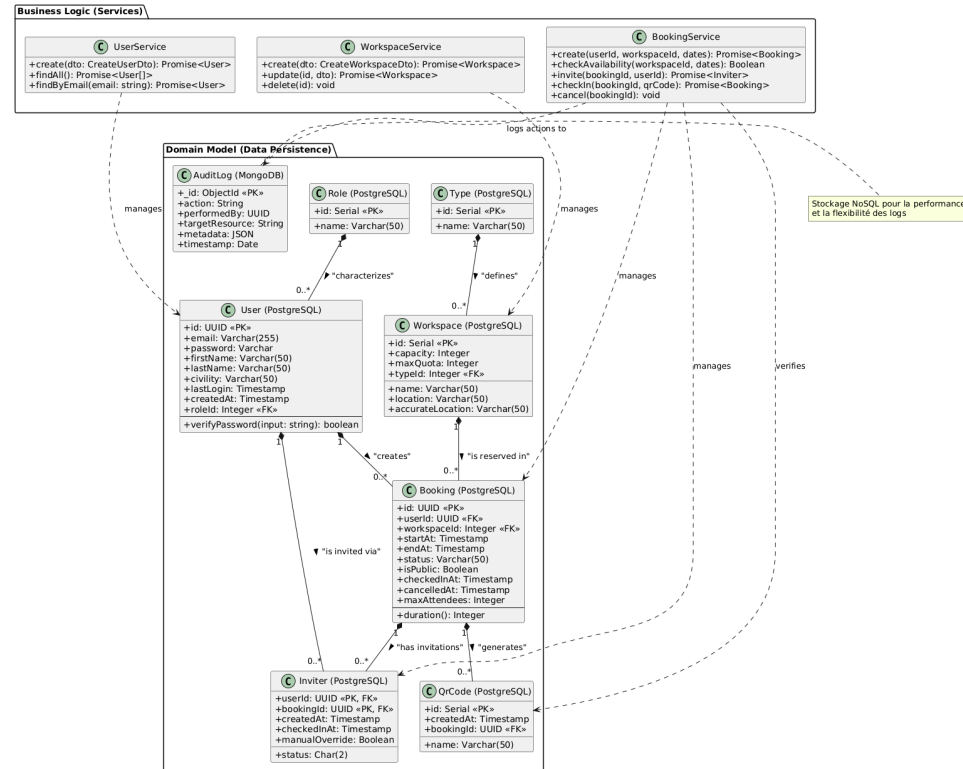
Du navigateur à la base

La requête traverse la route et le service avant l'écriture en base.

Dans la V1, le champ de visibilité est **visibility**. Une transaction crée la réservation et la participation de son propriétaire ; PostgreSQL protège les créneaux concurrents.

Ouvrir le diagramme complet dans un nouvel onglet

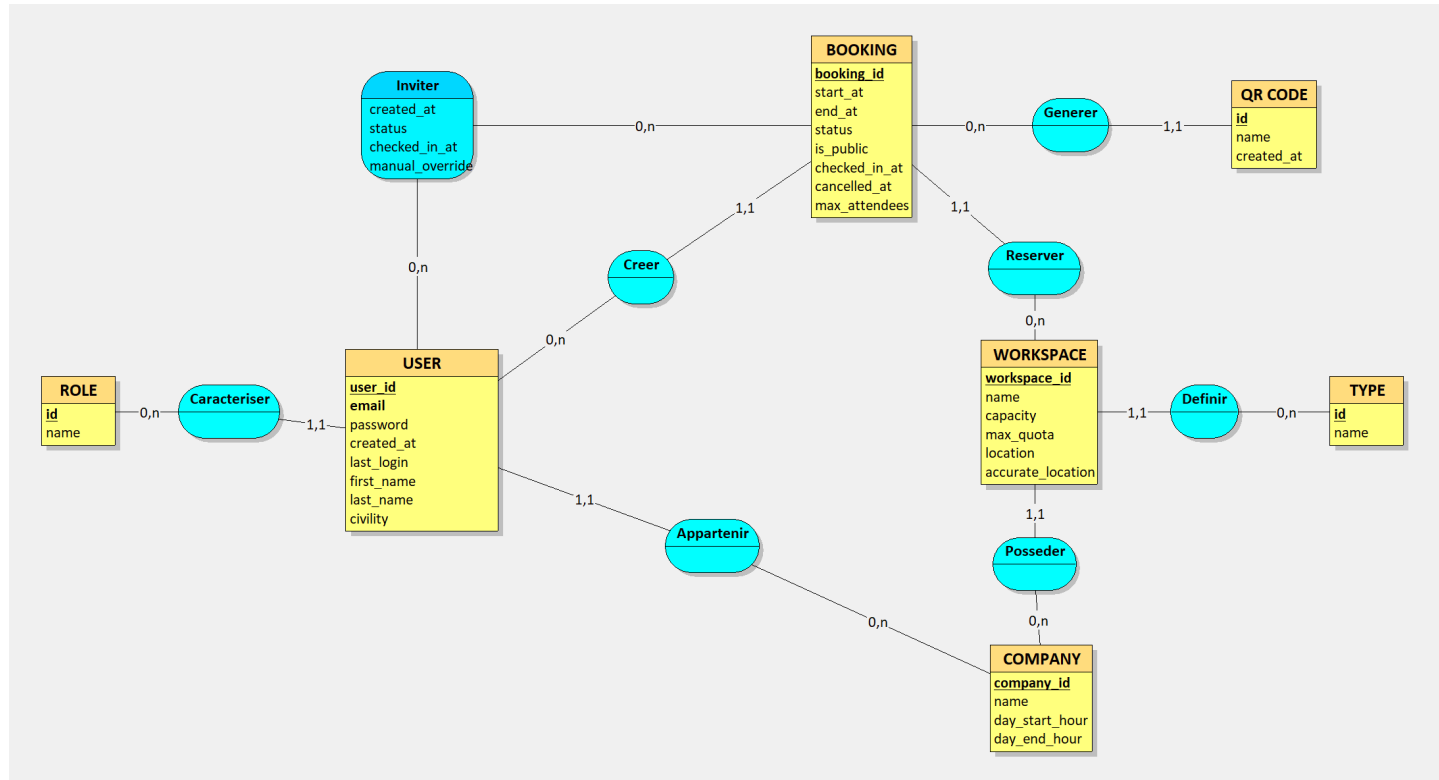
2.5. Diagramme de classes



Le modèle décrit la conception. Le code utilise surtout des services sous forme d'objets et de fonctions. Les quotas et plusieurs attributs du modèle ne font pas partie de la V1.

Ouvrir le diagramme complet dans un nouvel onglet

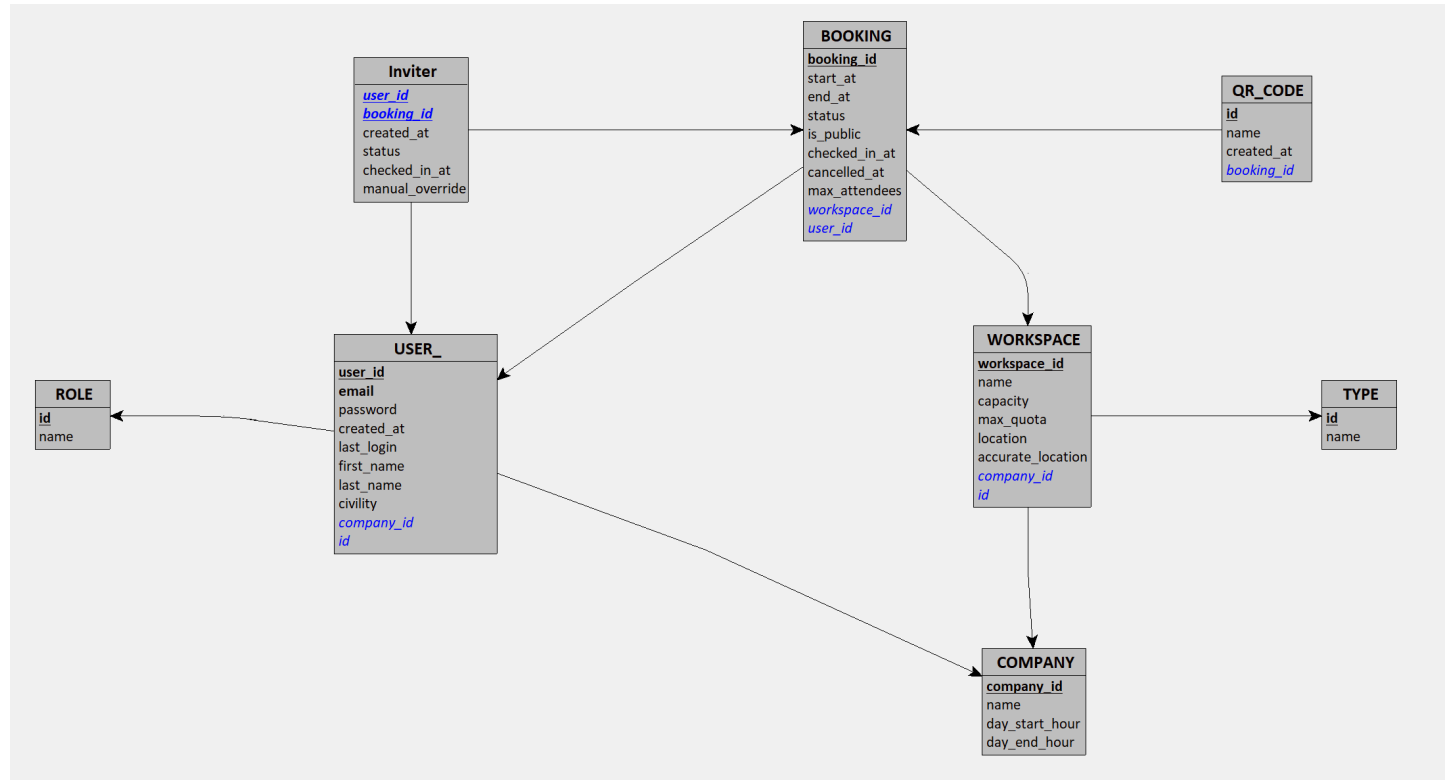
2.6. MERISE : le MCD



Le MCD décrit les entités et leurs liens métier. Le modèle initial inclut un périmètre plus large que la V1.

[Ouvrir le diagramme complet dans un nouvel onglet](#)

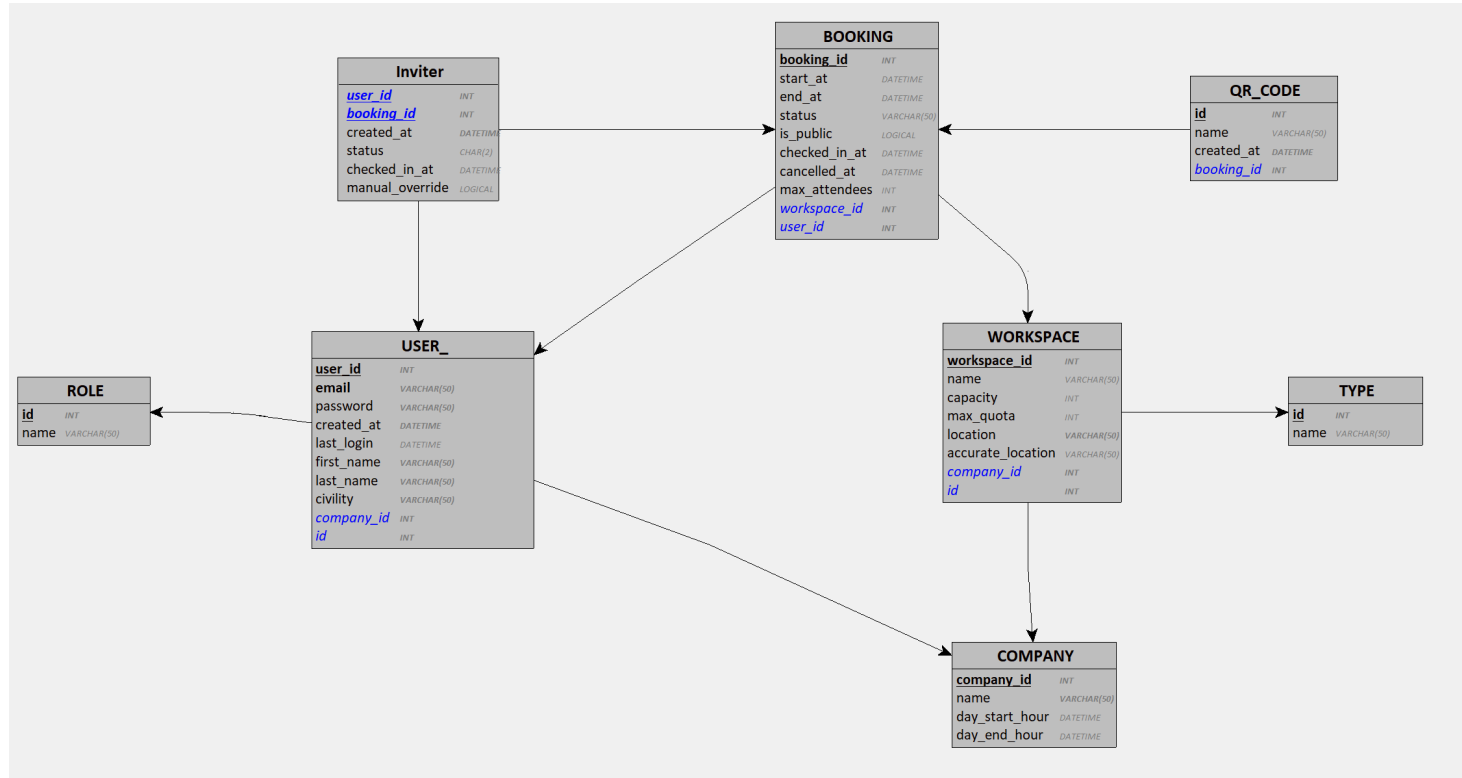
2.7. MERISE : le MLD



Dans le MLD, la table `Inviter` relie les utilisateurs aux réservations. Dans le code, cette relation est implémentée par `booking\_participants`, qui conserve le rôle du participant, sa réponse à l'invitation et l'heure de son check-in.

[Ouvrir le diagramme complet dans un nouvel onglet](#)

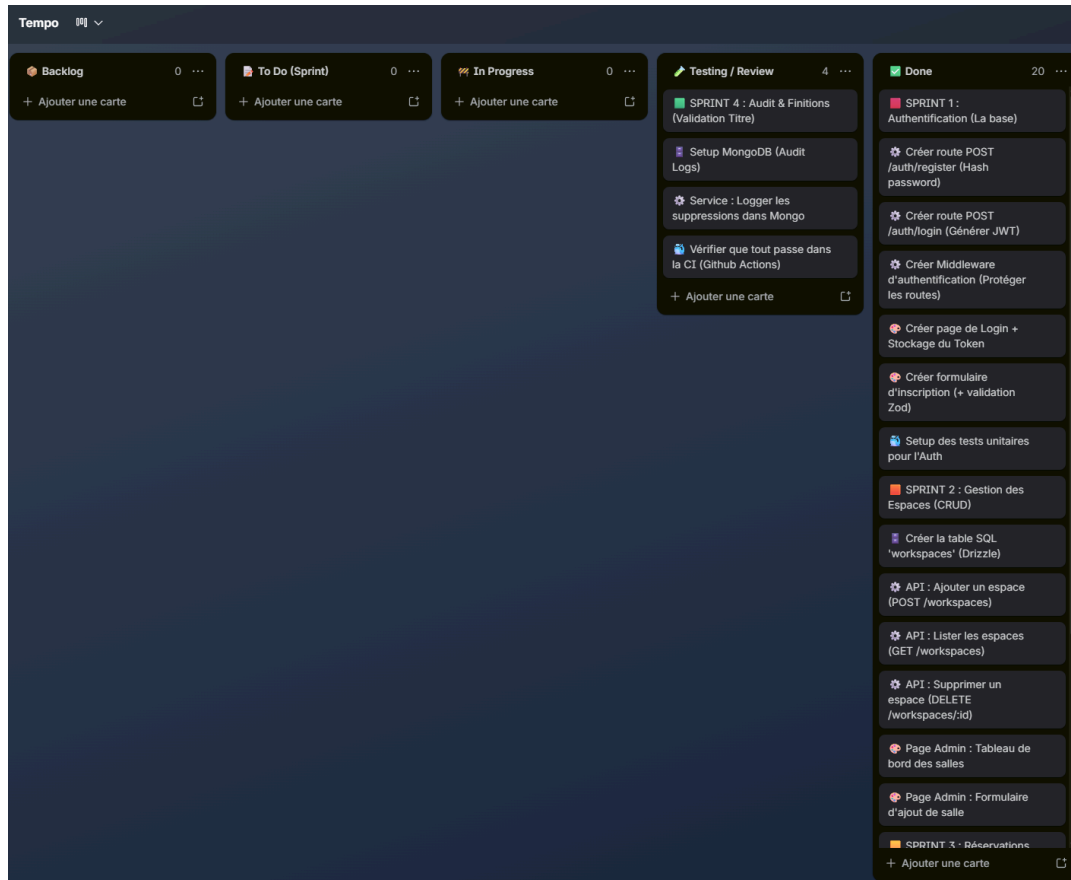
2.8. MERISE : le MPD



Le schéma Drizzle et les migrations décrivent la base mise en place : cinq tables PostgreSQL, des UUID pour les comptes et réservations, et des enums pour les rôles.

[Ouvrir le diagramme complet dans un nouvel onglet](#)

3. Méthodologie et organisation



- Un projet réalisé seul
- Des cartes dans « À faire », « En cours » et « Terminé »
- Des tests au fil du développement
- Git pour conserver l'historique

4. Architecture

Une application organisée en trois couches

4.1. Choix techniques

Outil	Utilisation dans Tempo
Svelte 5, SvelteKit, Tailwind et shadcn-svelte	Écrans et formulaires
Bun et Hono	API TypeScript et exécution des services
Drizzle et PostgreSQL	Données métier, transactions et contraintes
MongoDB	Journal des suppressions
Docker Compose	Environnement de démonstration

4.2. Architecture générale

Couche	Responsabilité
Présentation : SvelteKit	Afficher les pages et envoyer les demandes
API et services : Hono	Vérifier les droits et appliquer les règles
Données : PostgreSQL et MongoDB	Conserver les données métier et les audits

4.3. Architecture backend

- Modules `auth`, `users`, `workspaces`, `bookings`, `analytics` et `audit`
- Les routes reçoivent les requêtes et renvoient les statuts HTTP
- Les services appliquent les règles métier
- Drizzle et le pilote MongoDB exécutent les accès aux données
- Zod valide les entrées avant l'appel du service

4.4. Architecture frontend

- Pages SvelteKit et états locaux avec les runes Svelte
- Client Hono RPC typé avec `AppType`
- Redirections selon la session et le rôle
- Réponses 401 et 403 traitées dans le client

Sur mobile, les champs passent en colonne et les tableaux défilent dans leur propre zone.

Toutes les réservations
Supervisez les réservations de l'ensemble des utilisateurs

← Accueil Espaces Audit

admin@tempo.local ↗

Nouvelle Réservation

Choisir un espace

Privée

dd- ---- -yyyy

09:00

dd- ---- -yyyy

10:00

Réserver

2 réservations

Espace	Propriétaire	Dé
 Salle Atlas Salle - 6 pers.	admin@tempo.local ADMIN	15/
 RECETTE-2026-09-24 Bureau - 4 pers.	admin@tempo.local ADMIN	24/

5. Développement : créer une réservation

Toutes les réservations
Supervisez les réservations de l'ensemble des utilisateurs

← Accueil **Espaces** Audit admin@tempo.local →

Nouvelle Réservation

Erreur
Ce créneau est déjà réservé pour cet espace

RECETT... ▾

Privée ▾

24-Sep-2026

10:00

24-Sep-2026

11:00

Réserver

2 réservations

Espace	Propriétaire	Début	Fin	Visibilité	Participants	Statut	Actions
Salle Atlas Salle · 6 pers.	admin@tempo.local ADMIN	15/12/2027 14:00	15/12/2027 16:00	Publique	👤 2/6	À venir	Gérer Annuler
RECETTE-2026-09-24 Bureau · 4 pers.	admin@tempo.local ADMIN	24/09/2026 09:00	24/09/2026 18:00	Privée	👤 1/4	En cours	Gérer Annuler

1. Le formulaire envoie l'espace, les dates et la visibilité
2. L'API contrôle les données et la session
3. Le service tente la création
4. Le frontend affiche le résultat

5.1. La route reçoit la demande

```
.post('/', zValidator('json', createBookingSchema), async (c) => {  
  try {  
    const payload = c.get('jwtPayload');  
    const data = c.req.valid('json');  
  
    const booking = await bookingService.create(payload.sub, data);  
    return c.json(booking, 201);  
  }  
})
```

La route utilise l'identité de la session, puis transmet les données validées au service.

Extrait : apps/backend/src/modules/bookings/bookings.route.ts

5.2. Le service et la transaction

```
const hasOverlap = await this.checkOverlap(data.workspaceId, startAt, endAt);

if (hasOverlap) {
  throw new Error('BOOKING_OVERLAP');
}
```

Le service vérifie d'abord l'existence de l'espace et la disponibilité.

La transaction crée ensuite **la réservation et son propriétaire comme participant accepté.**

Extrait : `apps/backend/src/modules/bookings/bookings.service.ts`

5.3. PostgreSQL empêche le doublon

```
ALTER TABLE "bookings"  
ADD CONSTRAINT "bookings_workspace_time_exclusion"  
EXCLUDE USING gist (  
    "workspace_id" WITH =,  
    tsrange("start_at", "end_at", '[]') WITH &&  
);
```

- Deux requêtes concurrentes ne peuvent pas réserver le même espace au même moment
- L'intervalle `[]` permet deux créneaux consécutifs
- Le conflit devient une réponse **HTTP 409**

Migration : apps/backend/drizzle/0002_booking_overlap_constraint.sql

5.4. Invitation et check-in

Participants et QR code
RECETTE-2026-09-24 - 24/09/2026 09:00

Participants

admin@tempo.local Organisateur	Accepté
user@tempo.local Invité	En attente
participant@entreprise.fr	Inviter

Générer le QR code

Une invitation en attente occupe déjà une place.

Check-in Tempo



Présence confirmée

Check-in enregistré le 24/09/2026 10:35:58.

Retour aux réservations

Le participant accepté confirme sa présence pendant le créneau.

6. Sécurité : session et autorisations

```
const payload = await readSession(c);  
if (!payload) return c.json({ error: 'Authentication requise' }, 401);  
c.set('jwtPayload', payload);  
await next();
```

- JWT dans un cookie **HttpOnly**, `SameSite=Strict`, `Secure` en HTTPS
- Contrôle du rôle ADMIN et des droits sur chaque réservation
- Origine autorisée et en-tête CSRF pour les modifications

Extrait : apps/backend/src/middlewares/auth.guard.ts

6.1. Protection des données et du QR

- Mots de passe hachés avec `Bun.password`
- Validation Zod et contraintes PostgreSQL
- Limitation des tentatives d'inscription et de connexion
- Jeton QR aléatoire de 256 bits, seul son hash SHA-256 reste en base

7. Tests et qualité logicielle

Vérifier les règles, puis les parcours complets

7.1. Stratégie de test

Niveau	Ce que je vérifie
Unitaires et HTTP	Services, routes, rôles et erreurs
Frontend	Session, client RPC et gardes de navigation
Intégration PostgreSQL	Transactions, capacité et concurrence
Intégration MongoDB	Écriture et lecture des audits
Playwright	Réservation, invitation et check-in

7.2. Exemple de test unitaire

```
await expect(
  bookingService.create('user-1', {
    workspaceId: 1,
    startAt: '2024-01-01T10:30:00Z',
    endAt: '2024-01-01T11:30:00Z',
    visibility: 'PRIVATE',
  }),
).rejects.toThrow('BOOKING_OVERLAP');
```

Le test simule une réservation existante de **10 h à 12 h** et attend le refus du nouveau créneau.

Extrait : apps/backend/src/modules/bookings/bookings.service.spec.ts

7.3. Qualité

- Oxlint et Oxfmt contrôlent le code et son formatage
- TypeScript et Svelte Check détectent les erreurs de types
- Le build vérifie la compilation de production

8. CI/CD : pipeline de vérification

The screenshot displays a CI/CD pipeline interface. At the top, it shows the pipeline name 'docs: improve mobile responsiveness with reduced padding and vertical...' with a green checkmark and the number 78. Below this, a summary section indicates the pipeline was triggered via push yesterday by 'Vaalley pushed' on the 'main' branch, with a status of 'Success' and a total duration of '2m 45s'. The pipeline steps are listed as 'Quality & Tests' (40s) and 'Docker Build Check' (1m 57s). A 'Vitest Test Report' section provides a summary of test results: 4 test files, 4 passes, and 22 test results, all successful. The report also notes that the job summary was generated at run-time.

Quality & Tests

Format, lint, types, tests et build

Docker Build Check

Stack Docker, seed, intégrations
et E2E

Preuve datée du 23 septembre 2026,
commit `8d0d1cd`.

8.1. Déploiement de la démonstration

- Serveur Ubuntu partagé, fichiers Tempo dans `/opt/tempo/`
- Docker Compose pour les applications et les bases
- Caddy et DuckDNS pour l'accès HTTPS
- Mots de passe et clé JWT fournis au démarrage, sans les inclure dans les images Docker
- Bases sans exposition directe à Internet

CI automatisée, déploiement manuel

9. Bilan et perspectives

Ce que j'ai mis en place

- Le parcours de réservation jusqu'au check-in
- Des règles protégées jusque dans la base
- Une démonstration accessible en HTTPS

Ce qui reste à travailler

- Charge, accessibilité et autres navigateurs
- Intégration de fonctionnalités pour une V2
- Amélioration de l'expérience utilisateur

10. Conclusion et remerciements

Tempo m'a permis de travailler un projet complet, depuis le besoin jusqu'à sa mise en ligne.

Merci aux formateurs de la 3W Academy, à mon tuteur et à l'ami qui héberge la démonstration.

Merci pour votre attention.